Lehrerexemplar · Boss-Kampf 3 · Welt 3 · DS 23

Erwartungshorizont: Die Kristallhöhle

Erwartungshorizont / Bewertungsbogen für _____ Korrigiert von: _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen	I	II	III	BE
a) Dokumentation des Rekursionsverlaufs Kernpunkte: Zehn Aufrufe in der Reihenfolge ihres Beginns mit Rekursionstiefe: (0, 2) T0; (0, 1) T1; (0, 0) T2; (1, 1) T2; (0, 0) T2 mit 6, 9, 4; (2, 2) T1; (0, 1) T1 mit 6, 4, 9; (0, 0), (1, 1) T2; (0, 0) T2 mit 4, 6, 9; Belegung nach dem Ende 4, 6, 9.	2	2		
b) Anzahl der Aufrufe; Beurteilung der Effizienz Kernpunkte: n = 1: 1, n = 2: 4, n = 3: 10 (aus der Tabelle); n = 4: 1 + 4 + 4 + 10 = 19; Aussage falsch – ordne löst nach den beiden Hälften noch einmal ein Problem der Größe n − 1; Verdopplung 8 → 16 vervielfacht die Aufrufe etwa um den Faktor 10 (124 → 1219), bei Mergesort nur etwa um den Faktor 2 (15 → 31); Rekursionstiefe n − 1 statt etwa $\log_2$ n.	1	1	1	
c) Prinzip rekursiver Operationen; Begründung Kernpunkte: Rekursive Operation ruft sich selbst für ein kleineres Teilproblem auf; Basisfall li ≥ re (höchstens ein Element, ohne Selbstaufruf, nichts zu tun); Rekursionsschritt: Selbstaufrufe für die Bereiche li … m, m + 1 … re und li … re − 1, die jeweils echt weniger Elemente haben – der Basisfall wird sicher erreicht; Begründung: nach den beiden ersten Selbstaufrufen sind beide Hälften geordnet, ihr jeweils größter Wert steht an m bzw. an re; durch den Vergleich und ggf. Tausch steht das Maximum beider an re.	1	2		
d) Entwicklung eines rekursiven Struktogramms (Teile und herrsche) Kernpunkte: Kopf mit Signatur; Basisfall von = bis mit Rückgabe 1 bei r[von] ≥ grenze, sonst 0; mitte ← (von + bis) / 2; zwei Selbstaufrufe für von … mitte und mitte + 1 … bis (überschneidungsfrei, vollständig); Rückgabe der Summe der Teilergebnisse; keine Schleife; Notation nach EH §5.	1	2	1	
e) Rekursive Implementierung der Operation abbauen Kernpunkte: Methodenkopf public int abbauen(int z, int s); Abbruch mit Rückgabe 0, wenn (z|s) außerhalb liegt oder kein 'K' – Bereichsprüfung VOR dem Feldzugriff (Kurzschluss); Markieren mit '-' VOR den Selbstaufrufen; vier Selbstaufrufe (oben, rechts, unten, links); Rückgabe 1 + Summe der Teilergebnisse; Randfälle (Ader am Rand, abgebaute Felder) ohne Laufzeitfehler und ohne Endlosrekursion.	1	3	1	
f) Fehleranalyse und Korrektur eines rekursiven Struktogramms Kernpunkte: Fehler: Selbstaufruf finde(r, x, mid, hi) statt finde(r, x, mid + 1, hi); Verlauf (lo|hi|mid): (0|5|2) → (2|5|3) → (3|5|4) → (4|5|4) → (4|5|4) …; der Bereich wird nicht kleiner, derselbe Aufruf wiederholt sich – Endlosrekursion, Abbruch mit Stapelüberlauf (StackOverflowError), kein Rückgabewert; Korrektur mid + 1; Terminierung: mid liegt im Bereich und wird ausgeschlossen, jeder Selbstaufruf verkleinert hi − lo + 1 echt, nach endlich vielen Schritten Treffer oder lo > hi.	1	3	1	
Summe	7	13	4	**/ 24**

AFB-Verteilung: I 7 BE (29 %) · II 13 BE (54 %) · III 4 BE (17 %) – Abitur-Richtwert 30 / 53 / 17 %. Zeitbudget: 24 BE · 2,5 min = 60 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten.

Auswertung und Algo Quest Lab (fließt nicht in die Note ein)

- Der Papier-Boss ist **handschriftliches Abi-Training**: Auswertung durch die Lehrkraft oder Selbst-/Peer-Korrektur nach diesem Erwartungshorizont. Punkte werden dafür nicht vergeben – Punkte entstehen nur automatisch im Algo Quest Lab.
- Papier-Compiler zu e): Handschrift unverändert abgetippt, alle 8 Tests grün – Ergebnis im Antwortbogen ankreuzen (beim 1. Versuch / nach Korrektur / nicht grün).
- **Fehlerjagd** – darauf besonders achten: In e) Bereichsprüfung vor jedem Feldzugriff und Markierung vor den Selbstaufrufen (Tests aderAmRand und grosseAder grün), in f) Korrektur mid + 1 mit Begründung.
- **Zusätzlich im Lab: Online-Boss „Das Echo der Schlucht“** (25 min, 8 Aufgaben, Auswertung am Ende, der beste Versuch zählt, „besiegt“ ab 50 %). Punkte und Abzeichen („Das Echo der Schlucht besiegt“, ggf. „Makellos“) vergibt das Lab automatisch; die Ergebnisse stehen in der Kurs-Auswertung.
- Orientierung (informativ, Abi-Skala): ab 95 % 15 P · 85 % 13 P · 75 % 11 P · 65 % 9 P · 55 % 7 P · 45 % 5 P · 40 % 4 P · 27 % 2 P. Für 24 BE: 23 BE ≈ 15 P, 18 BE ≈ 11 P, 10 BE ≈ 4 P.

Musterlösungen, Kernpunkte und typische Fehler

a) Dokumentation des Rekursionsverlaufs 4 BE (I 2 · II 2 · III 0)

Musterlösung

Rekursionsverlauf (geprüft durch Ausführen einer zeilengetreuen, instrumentierten Java-Fassung von Abbildung 3):

Rekursionstiefe *)	Aufruf	r[0]	r[1]	r[2]
0	ordne(r, 0, 2)	9	6	4
1	ordne(r, 0, 1)	9	6	4
2	ordne(r, 0, 0)	9	6	4
2	ordne(r, 1, 1)	9	6	4
2	ordne(r, 0, 0)	6	9	4
1	ordne(r, 2, 2)	6	9	4
1	ordne(r, 0, 1)	6	4	9
2	ordne(r, 0, 0)	6	4	9
2	ordne(r, 1, 1)	6	4	9
2	ordne(r, 0, 0)	4	6	9

*) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.

Belegung nach dem Ende des Erstaufrufs: **4, 6, 9**. Getauscht wird dreimal: in ordne(r, 0, 1) (9 ↔ 6), im Erstaufruf nach den Hälften (9 ↔ 4) und im letzten ordne(r, 0, 1) (6 ↔ 4).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Aufrufe der Tiefen 0 und 1 vollständig und in richtiger Reihenfolge	I	1	☐
Aufrufe der Tiefe 2 (auch die sofort endenden mit li = re)	I	1	☐
Belegungen zum Zeitpunkt der Aufrufe (Tausch erst nach den Hälften)	II	1	☐
Endbelegung 4, 6, 9 und konsistente Rekursionstiefen	II	1	☐

Alternativen / Teilpunkte: Folgefehler werden nicht erneut abgezogen. Werden die sofort endenden Aufrufe mit li = re vergessen, höchstens 2 BE. Die Schreibweise ordne(vitrine, …) statt ordne(r, …) ist gleichwertig.

Typische Fehler

- Aufrufe mit li = re weggelassen („da passiert ja nichts“).
- Belegung nach statt vor dem Aufruf notiert.
- Rekursionstiefe der Geschwisteraufrufe erhöht (2 → 3) statt gleich gelassen.
- Letzten Selbstaufruf ordne(r, li, re − 1) übersehen.

b) Anzahl der Aufrufe; Beurteilung der Effizienz 3 BE (I 1 · II 1 · III 1)

Musterlösung

Aus a): n = 1 (z. B. ordne(r, 2, 2)): **1** Aufruf · n = 2 (ordne(r, 0, 1) mit zwei Aufrufen für li = re und einem für re − 1): **4** · n = 3 (Erstaufruf): **10**.

n = 4: Für ordne(r, 0, 3) ist m = 1. Es folgen ordne(r, 0, 1) (2 Elemente, 4 Aufrufe), ordne(r, 2, 3) (2 Elemente, 4 Aufrufe) und ordne(r, 0, 2) (3 Elemente, 10 Aufrufe): 1 + 4 + 4 + 10 = **19**. Allgemein: $A(n) = 1 + A(\lceil n/2 \rceil) + A(\lfloor n/2 \rfloor) + A(n - 1)$. Alle Werte durch Ausführen geprüft (n = 5: 34, n = 8: 124, n = 16: 1219).

Beurteilung: Die Aussage ist falsch. ordne teilt zwar in zwei Hälften, „herrscht“ aber nicht durch einfaches Zusammenführen (wie verschmelze), sondern ruft sich danach noch einmal für fast den ganzen Bereich (n − 1 Elemente) auf. Das Problem schrumpft dadurch nur um ein Element pro Ebene (Rekursionstiefe n − 1). Bei Verdopplung von 8 auf 16 Elemente steigt die Zahl der Aufrufe auf etwa das Zehnfache (124 → 1219), bei Mergesort nur auf etwa das Doppelte (15 → 31). ordne ist also sehr viel ineffizienter – das Teilen allein macht ein Verfahren nicht effizient.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Anzahlen 1, 4, 10 aus der Dokumentation	I	1	☐
Ermittlung von 19 Aufrufen für n = 4 (nachvollziehbar)	II	1	☐
Begründete Beurteilung (zusätzlicher Aufruf für n − 1, Wachstum bei Verdopplung)	III	1	☐

Alternativen / Teilpunkte: Für n = 4 ist auch ein vollständig gezeichneter Aufrufbaum oder eine Tabelle gleichwertig. Die Beurteilung kann ohne Formel allein über die angegebenen Zahlen (Faktor ≈ 10 statt ≈ 2) begründet werden; der Hinweis auf die Tiefe n − 1 ist nicht gefordert.

Typische Fehler

- Erstaufruf nicht mitgezählt (0, 3, 9, 18).
- n = 4 durch Verdopplung „geschätzt“ (20) statt ermittelt.
- „Beide halbieren, also gleich schnell“ – der dritte Selbstaufruf wird übersehen.
- Beurteilung nur über die Laufzeit „in Sekunden“ ohne Bezug zu den Zahlen.

c) Prinzip rekursiver Operationen; Begründung 3 BE (I 1 · II 2 · III 0)

Musterlösung

Prinzip: Eine rekursive Operation löst ein Problem, indem sie sich selbst für kleinere Teilprobleme derselben Art aufruft. ordne hat den **Basisfall** li ≥ re: Ein Bereich mit höchstens einem Element ist bereits geordnet, es erfolgt kein Selbstaufruf. Im **Rekursionsschritt** (li < re) ruft sich ordne für die Bereiche li … m, m + 1 … re und li … re − 1 auf; jeder hat echt weniger Elemente als li … re (m < re wegen der Division ohne Rest). Deshalb wird der Basisfall nach endlich vielen Schritten erreicht und die Aufrufe kehren nacheinander zurück.

Begründung: Nach ordne(r, li, m) ist die linke Hälfte geordnet, ihr größter Wert steht ganz rechts an Position m. Nach ordne(r, m + 1, re) steht der größte Wert der rechten Hälfte an Position re. Der größte Wert des ganzen Bereichs ist einer dieser beiden Werte. Die Verzweigung tauscht r[m] und r[re], falls r[m] größer ist – danach steht in jedem Fall der größere der beiden, also das Maximum des Bereichs, an re. (Deshalb genügt danach ordne(r, li, re − 1).)

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Selbstaufruf, Basisfall li ≥ re ohne Selbstaufruf	I	1	☐
Rekursionsschritt verkleinert den Bereich, Basisfall wird erreicht	II	1	☐
Begründung über die Maxima der geordneten Hälften an m und re	II	1	☐

Alternativen / Teilpunkte: Die Erläuterung über den Aufrufstapel (jeder Aufruf hat eigene Werte von li, re und m und wartet auf die Rückkehr der Selbstaufrufe) ist gleichwertig zum Argument der Verkleinerung. Basisfall „li = re“ ist gleichwertig (li > re kommt bei ordne nicht vor). Eine Begründung am Beispiel aus a) allein erhält höchstens 1 BE für den zweiten Teil.

Typische Fehler

- Basisfall nicht benannt oder mit einem Selbstaufruf verwechselt („Basisfall: ordne(r, li, m)“).
- „ordne ruft sich auf, bis alles sortiert ist“ – ohne Basisfall und Verkleinerung.
- Begründung „weil getauscht wird“, ohne zu sagen, warum r[m] und r[re] die beiden Kandidaten für das Maximum sind.

d) Entwicklung eines rekursiven Struktogramms (Teile und herrsche) 4 BE (I 1 · II 2 · III 1)

Musterlösung

Musterlösung (in Java übersetzt und geprüft: Abbildung 1 mit grenze 10 → 4, grenze 13 → 1; 3000 Zufallsfälle gegen Zählschleife):

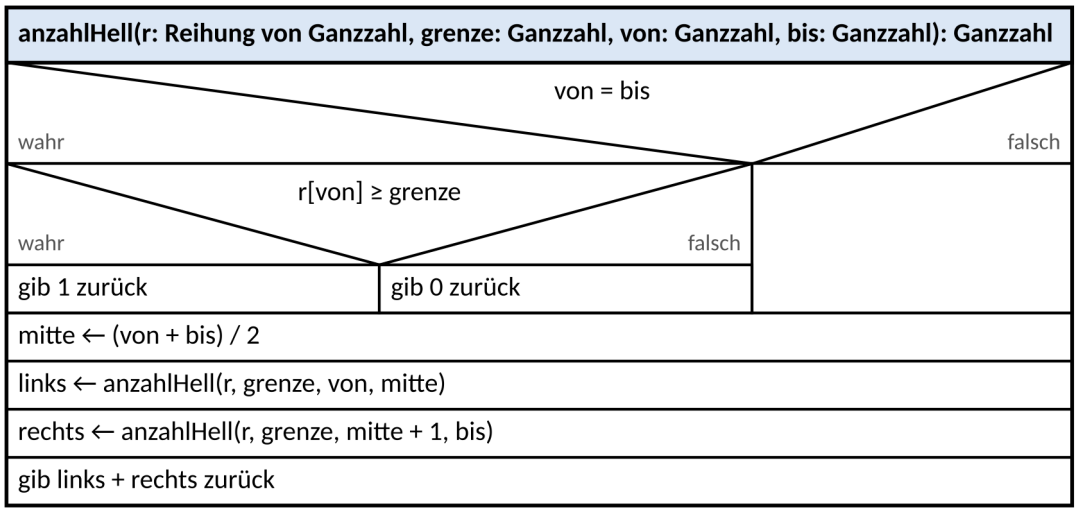


Abbildung EH.1: Struktogramm der Operation anzahlHell

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Kopf mit Signatur, Rückgabe in jedem Pfad	I	1	☐
Basisfall von = bis mit Vergleich ≥ grenze (1 bzw. 0)	II	1	☐
Teilung in der Mitte und zwei Selbstaufrufe mit passenden Parametern	II	1	☐
Teilbereiche überschneidungsfrei und vollständig (mitte + 1), Zusammenführen durch Addition	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: Zuweisungen der Teilergebnisse weglassen und „gib anzahlHell(r, grenze, von, mitte) + anzahlHell(r, grenze, mitte + 1, bis) zurück“; Basisfall als „falls von = bis und r[von] ≥ grenze“ mit getrenntem Fall für 0. Eine lineare Rekursion (z. B. r[von] prüfen und anzahlHell(r, grenze, von + 1, bis)) ist korrekt, folgt aber nicht dem geforderten Prinzip: höchstens 2 BE. Lösungen mit Schleife: höchstens 1 BE.

Typische Fehler

- Teilung von … mitte − 1 und mitte … bis mit mitte = (von + bis) / 2 → bei zwei Elementen ist der linke Teil leer (von > bis) bzw. Endlosrekursion.
- Beide Teile mit mitte (r[mitte] doppelt gezählt).
- > statt ≥ (Kristalle mit genau grenze fehlen).
- Basisfall fehlt oder liefert r[von] statt 1/0.
- Java-Syntax (==, return) statt EH-Notation.

e) Rekursive Implementierung der Operation abbauen 5 BE (I 1 · II 3 · III 1)

Musterlösung

Musterlösung (Java, mit javac und den Papier-Compiler-Tests geprüft: 8/8 grün; abbauen(2, 3) = 6, abbauen(0, 0) = 4, abbauen(3, 0) = 1):

```
public int abbauen(int z, int s) {
    if (!istImFeld(z, s) || feld[z][s] != 'K') {
        return 0;
    }
    feld[z][s] = '-';
    return 1 + abbauen(z - 1, s) + abbauen(z, s + 1)
           + abbauen(z + 1, s) + abbauen(z, s - 1);
}
```

Höhle nach abbauen(2, 3):

	s = 0	s = 1	s = 2	s = 3	s = 4	s = 5
z = 0	K	K	#	.	-	-
z = 1	#	K	.	.	#	-
z = 2	.	K	#	-	-	-
z = 3	K	.	.	#	.	.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf; Rückgabe einer Ganzzahl in jedem Pfad	I	1	☐
Abbruch außerhalb bzw. ohne Kristall; Bereichsprüfung vor dem Feldzugriff	II	1	☐
Markierung '-' vor den Selbstaufrufen	II	1	☐
Vier Selbstaufrufe in der 4er-Nachbarschaft, Summe + 1	II	1	☐
Randfälle ohne Laufzeitfehler, keine Endlosrekursion; nur die Ader wird verändert	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (beide ebenfalls 8/8 Tests grün): (1) Bereichsprüfung ohne istImFeld mit vier Vergleichen und Zählvariable anzahl; (2) Verschiebungen in Hilfsreihungen und Schleife über die vier Richtungen. Ebenso richtig: vor jedem Selbstaufruf prüfen, ob der Nachbar ein Kristall ist (dann muss auch der Erstaufruf geprüft werden). Struktogramm statt Java: höchstens 3 BE (vgl. Abbildung EH.2). Kleinere Syntaxfehler führen nicht zu Abzug, wenn die Logik eindeutig ist – beim Papier-Compiler zählen sie aber!

Alternative (1): Bereichsprüfung ohne istImFeld, Zählvariable

```
public int abbauen(int z, int s) {
    if (z < 0 || z >= feld.length || s < 0 || s >= feld[0].length) {
        return 0;
    }
    if (feld[z][s] != 'K') {
        return 0;
    }
    feld[z][s] = '-';
    int anzahl = 1;
    anzahl = anzahl + abbauen(z - 1, s);
    anzahl = anzahl + abbauen(z, s + 1);
    anzahl = anzahl + abbauen(z + 1, s);
    anzahl = anzahl + abbauen(z, s - 1);
    return anzahl;
}
```

Alternative (2): Richtungen in Hilfsreihungen

```
public int abbauen(int z, int s) {
    if (!istImFeld(z, s) || feld[z][s] != 'K') {
        return 0;
    }
    feld[z][s] = '-';
    int anzahl = 1;
    int[] dz = {-1, 0, 1, 0};
    int[] ds = {0, 1, 0, -1};
    for (int i = 0; i < 4; i++) {
        anzahl = anzahl + abbauen(z + dz[i], s + ds[i]);
    }
    return anzahl;
}
```

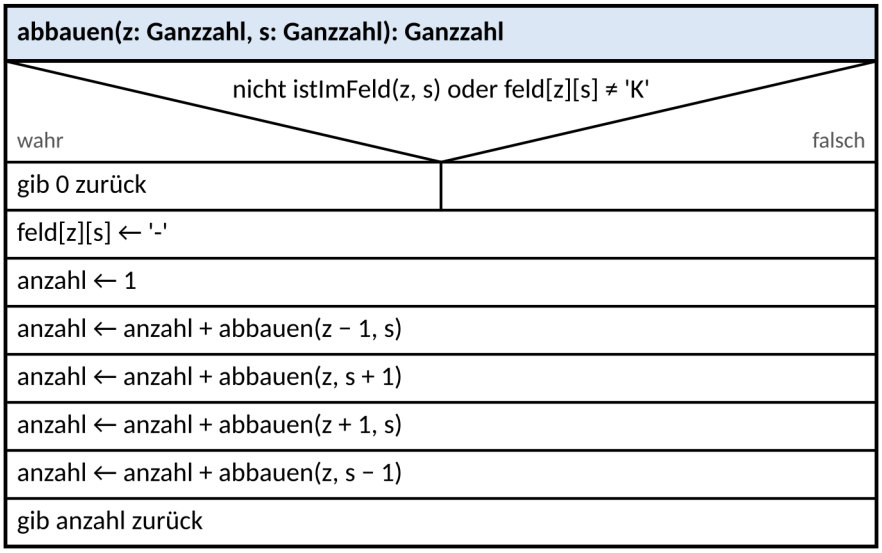


Abbildung EH.2: abbauen als Struktogramm (bei Abgabe als Struktogramm höchstens 3 BE)

Typische Fehler

- Feldzugriff vor der Bereichsprüfung (feld[z][s] != 'K' || !istImFeld(z, s)) → ArrayIndexOutOfBoundsException am Rand (Test aderAmRand rot).
- Markierung erst nach den Selbstaufrufen oder gar nicht → Endlosrekursion zwischen zwei Nachbarn (StackOverflowError).
- Diagonale Nachbarn mitgezählt.
- Eigener Kristall nicht mitgezählt (return abbauen(…) + … ohne 1).
- Nur auf Fels geprüft – Gänge werden „abgebaut".
- Zählvariable als Attribut statt lokal bzw. Rückgabewerte der Selbstaufrufe ignoriert.

f) Fehleranalyse und Korrektur eines rekursiven Struktogramms 5 BE (I 1 · II 3 · III 1)

Musterlösung

Verlauf des Aufrufs finde(lager, 17, 0, 5) (geprüft durch Ausführen; Tiefe | lo | hi | mid | r[mid]):

Tiefe	lo	hi	mid	r[mid]	Folge
0	0	5	2	8	r[mid] < 17 → finde(r, 17, 2, 5)
1	2	5	3	11	r[mid] < 17 → finde(r, 17, 3, 5)
2	3	5	4	14	r[mid] < 17 → finde(r, 17, 4, 5)
3	4	5	4	14	finde(r, 17, 4, 5) – derselbe Aufruf
4	4	5	4	14	finde(r, 17, 4, 5) – derselbe Aufruf
…	4	5	4	14	… ohne Ende

Der Fehler liegt im wahr-Zweig der Bedingung r[mid] < x: Der Selbstaufruf finde(r, x, mid, hi) schließt mid nicht aus. Sobald ein Bereich mit zwei Elementen erreicht ist (lo = 4, hi = 5), ist mid = (4 + 5) / 2 = 4 = lo; da r[4] = 14 < 17, folgt erneut finde(r, 17, 4, 5) – der Bereich wird nicht kleiner. Es entsteht eine **Endlosrekursion**; der Aufrufstapel läuft über, das Programm bricht mit einem Laufzeitfehler (StackOverflowError) ab, ein Rückgabewert wird nie geliefert. Bei 14 tritt der Fehler nicht auf, weil 14 schon im Bereich 3 … 5 an mid = 4 gefunden wird.

Betroffen sind (geprüft für x = 0 bis 20) alle gesuchten Werte, bei denen ein Zweierbereich mit r[lo] < x erreicht wird, z. B. 17 und alle fehlenden Werte über 2: 3, 4, 5, 6, 7, 9, 10, 12, 13, 15, 16, 17, 18, 19, 20.

Korrektur: Im wahr-Zweig von r[mid] < x wird „gib finde(r, x, mid + 1, hi) zurück" eingetragen (Abbildung EH.3).

Begründung der Terminierung: Bei jedem Aufruf mit lo ≤ hi liegt mid im Bereich lo … hi. Der Selbstaufruf erfolgt entweder mit mid + 1 … hi oder mit lo … mid − 1; beide Bereiche enthalten mid nicht mehr und haben deshalb mindestens ein Element weniger. Da die Anzahl hi − lo + 1 nicht beliebig oft kleiner werden kann, wird nach endlich vielen Aufrufen x gefunden oder der Basisfall lo > hi erreicht.

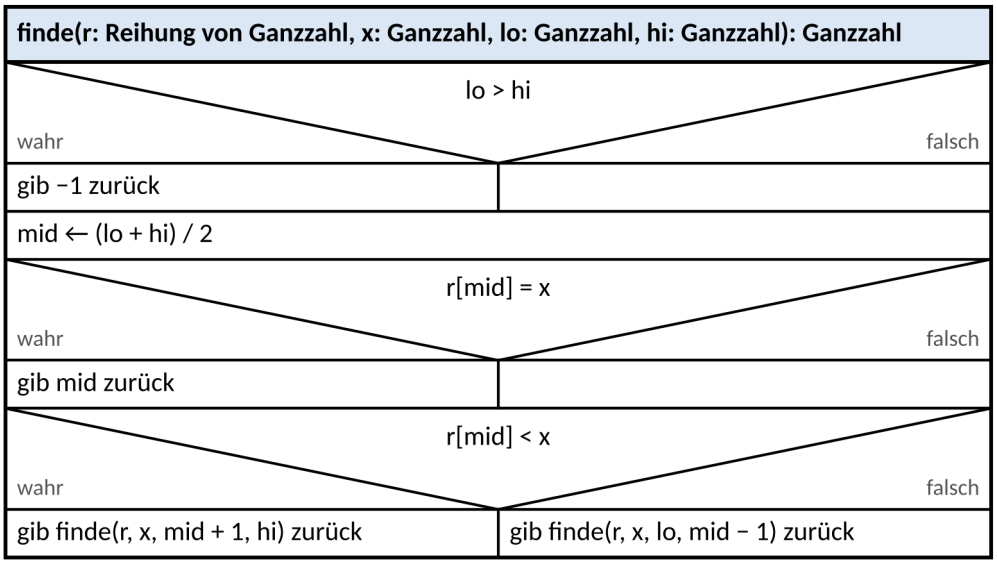


Abbildung EH.3: korrigiertes Struktogramm der Operation finde (geprüft: korrekt für alle Werte, auch bei 0 bis 40 Elementen)

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Fehlerhafte Stelle (Selbstaufruf mit mid statt mid + 1) identifiziert	I	1	☐
Verlauf für x = 17 bis zur Wiederholung (4, 5) dokumentiert	II	1	☐
Folge: Bereich wird nicht kleiner → Endlosrekursion, Stapelüberlauf, keine Rückgabe	II	1	☐
Korrektur im Struktogramm (mid + 1)	II	1	☐
Begründung der Terminierung über die echte Verkleinerung des Bereichs	III	1	☐

Alternativen / Teilpunkte: „Programm stürzt ab“ bzw. „hängt“ ist als Folge nur mit Begründung (gleicher Aufruf, Bereich wird nicht kleiner) ausreichend. Eine Darstellung des Verlaufs als Aufrufkette ohne Tabelle ist gleichwertig. Zusätzliche Änderungen, die korrekt bleiben (z. B. Basisfall lo > hi unverändert, zusätzliche Abfrage lo = hi), sind zulässig.

Typische Fehler

- Fehler beim Basisfall gesucht („lo > hi müsste lo ≥ hi heißen“) – das verschlimmert den Fehler.
- Korrektur mid − 1 im falschen Zweig oder beide Zweige geändert.
- „17 wird nicht gefunden, also −1“ statt Endlosrekursion.
- Terminierung nur behauptet („weil mid + 1“) ohne Verkleinerung des Bereichs.

Anhang: Papier-Compiler-Workspace zu e)

Für den Papier-Compiler legst du in der Online-IDE einen Workspace „W3 Boss 3 – Papier-Compiler“ mit der Klasse Hoehle (Gerüst unten, abbauen mit leerem Rumpf und return 0;) und der Testklasse BossTest an und verteilst ihn über „An Klasse austeilen“. Die Lernenden tippen ihre Handschrift **unverändert** in den Rumpf ab und starten die Tests im Reiter „Testrunner“. Erst danach dürfen Fehler korrigiert werden (dann im Antwortbogen „grün nach Korrektur“ ankreuzen).

Vor Einsatz prüfen

JUnit-Dialekt der Online-IDE (parameterloser Konstruktor der Testklasse, @Test, globale assertEquals mit Meldung als letztem Argument) vor dem ersten Einsatz mit einem Probedurchlauf testen – insbesondere 2D-Reihungen (new char[z][s], feld[z][s]), assertEquals mit Zeichen ('-') und mit Zeichenketten, die \n enthalten. Wichtig: Wie die Online-IDE eine Endlosrekursion meldet (Stapelüberlauf als Laufzeitfehler, ggf. Abbruch nach Zeitlimit), mit der Fehlvariante „Markierung erst nach den Selbstaufrufen“ ausprobieren – der Testrunner muss danach weiterlaufen. Die Hilfsmethode hoehle() der Testklasse ist private und ohne @Test. Die Bibliothek „niedersachsen“ wird nicht benötigt.

Klasse Hoehle (Gerüst für den Workspace; Konstruktor, istImFeld, getFeld und alsText sind vorhanden)

```
public class Hoehle {

    private char[][] feld;

    public Hoehle(String[] zeilen) {
        feld = new char[zeilen.length][zeilen[0].length()];
        for (int z = 0; z < zeilen.length; z++) {
            for (int s = 0; s < zeilen[z].length(); s++) {
                feld[z][s] = zeilen[z].charAt(s);
            }
        }
    }

    public boolean istImFeld(int z, int s) {
        return z >= 0 && z < feld.length && s >= 0 && s < feld[0].length;
    }

    public char getFeld(int z, int s) {
        return feld[z][s];
    }

    public String alsText() {
        String t = "";
        for (int z = 0; z < feld.length; z++) {
            if (z > 0) {
                t = t + "\n";
            }
            for (int s = 0; s < feld[z].length; s++) {
                t = t + feld[z][s];
            }
        }
        return t;
    }

    public int abbauen(int z, int s) {
        // HIER die Handschrift aus Aufgabe e) UNVERAENDERT abtippen
        return 0;
    }
}
```

Testklasse BossTest (8 Tests; mit der Musterlösung und beiden Alternativen alle grün, typische Fehler werden erkannt – geprüft mit javac, siehe Tabelle)

```
public class BossTest {

    public BossTest() {
    }

    private Hoehle hoehle() {
        return new Hoehle(new String[] {"KK#.KK", "#K..#K", ".K#KKK", "K..#.."});
    }

    @Test
    public void beispielAder() {
        Hoehle h = hoehle();
        assertEquals(6, h.abbauen(2, 3), "Ader ab (2|3) hat 6 Kristalle");
        assertEquals("KK#.--\n#K..#-\n.K#---\nK..#..", h.alsText(),
                "genau die 6 Kristalle der Ader sind abgebaut");
    }

    @Test
    public void aderAmRand() {
        Hoehle h = hoehle();
        assertEquals(4, h.abbauen(0, 0), "Ader in der Ecke (0|0) hat 4 Kristalle");
        assertEquals("--#.KK\n#-..#K\n.-#KKK\nK..#..", h.alsText(),
                "Ader am Rand vollstaendig abgebaut");
    }

    @Test
    public void einzelnerKristall() {
        Hoehle h = hoehle();
        assertEquals(1, h.abbauen(3, 0), "einzelner Kristall in der Ecke (3|0)");
        assertEquals('-', h.getFeld(3, 0), "Kristall (3|0) ist abgebaut");
        assertEquals('.', h.getFeld(2, 0), "Gang (2|0) bleibt Gang");
    }

    @Test
    public void keinKristall() {
        Hoehle h = hoehle();
        assertEquals(0, h.abbauen(1, 2), "Gang liefert 0");
        assertEquals(0, h.abbauen(0, 2), "Fels liefert 0");
        assertEquals("KK#.KK\n#K..#K\n.K#KKK\nK..#..", h.alsText(), "Hoehle unveraendert");
    }

    @Test
    public void ausserhalbDerHoehle() {
        Hoehle h = hoehle();
        assertEquals(0, h.abbauen(-1, 0), "oberhalb der Hoehle");
        assertEquals(0, h.abbauen(4, 0), "unterhalb der Hoehle");
        assertEquals(0, h.abbauen(0, 6), "rechts der Hoehle");
        assertEquals(0, h.abbauen(0, -1), "links der Hoehle");
        assertEquals("KK#.KK\n#K..#K\n.K#KKK\nK..#..", h.alsText(), "Hoehle unveraendert");
    }

    @Test
    public void zweimalAbbauen() {
        Hoehle h = hoehle();
        h.abbauen(2, 3);
        assertEquals(0, h.abbauen(2, 4), "abgebauter Kristall zaehlt nicht mehr");
```

```
        assertEquals(0, h.abbauen(2, 3), "zweiter Aufruf liefert 0");
    }

    @Test
    public void diagonalZaehltNicht() {
        Hoehle h = new Hoehle(new String[] {"K.K", ".K.", "K.K"});
        assertEquals(1, h.abbauen(1, 1), "diagonale Kristalle gehoeren nicht zur Ader");
        assertEquals('K', h.getFeld(0, 0), "diagonaler Kristall bleibt");
    }

    @Test
    public void grosseAder() {
        String[] z = new String[10];
        for (int i = 0; i < 10; i++) {
            z[i] = "KKKKKKKKKK";
        }
        Hoehle h = new Hoehle(z);
        assertEquals(100, h.abbauen(4, 5), "10 x 10 Kristalle");
        assertEquals(0, h.abbauen(0, 0), "alles abgebaut");
    }
}
```

Kontrolle der Testklasse mit typischen Fehlvarianten (Mutanten)

Variante von abbauen	Tests grün
Musterlösung (eine Abfrage, Summe der vier Selbstaufrufe)	8/8
Alternative (1): Bereichsprüfung ohne istImFeld, Zählvariable anzahl	8/8
Alternative (2): Verschiebungen in Hilfsreihungen dz/ds, Schleife über vier Richtungen	8/8
keine Bereichsprüfung (nur feld[z][s] != 'K')	2/8
Bereichsprüfung nach dem Feldzugriff (Reihenfolge vertauscht)	2/8
Markierung erst nach den Selbstaufrufen (Endlosrekursion)	4/8
Kristall wird nicht markiert (Endlosrekursion)	3/8
8er-Nachbarschaft (auch diagonal)	5/8
nur zwei Richtungen (rechts, unten)	6/8
eigener Kristall nicht mitgezählt (+1 fehlt)	3/8
prüft nur auf Fels (baut auch Gänge ab)	3/8
leerer Rumpf (Gerüst, return 0;)	3/8